

KATKEE

10/10 FINAL MASTER PRODUCT +

CODEX SPEC

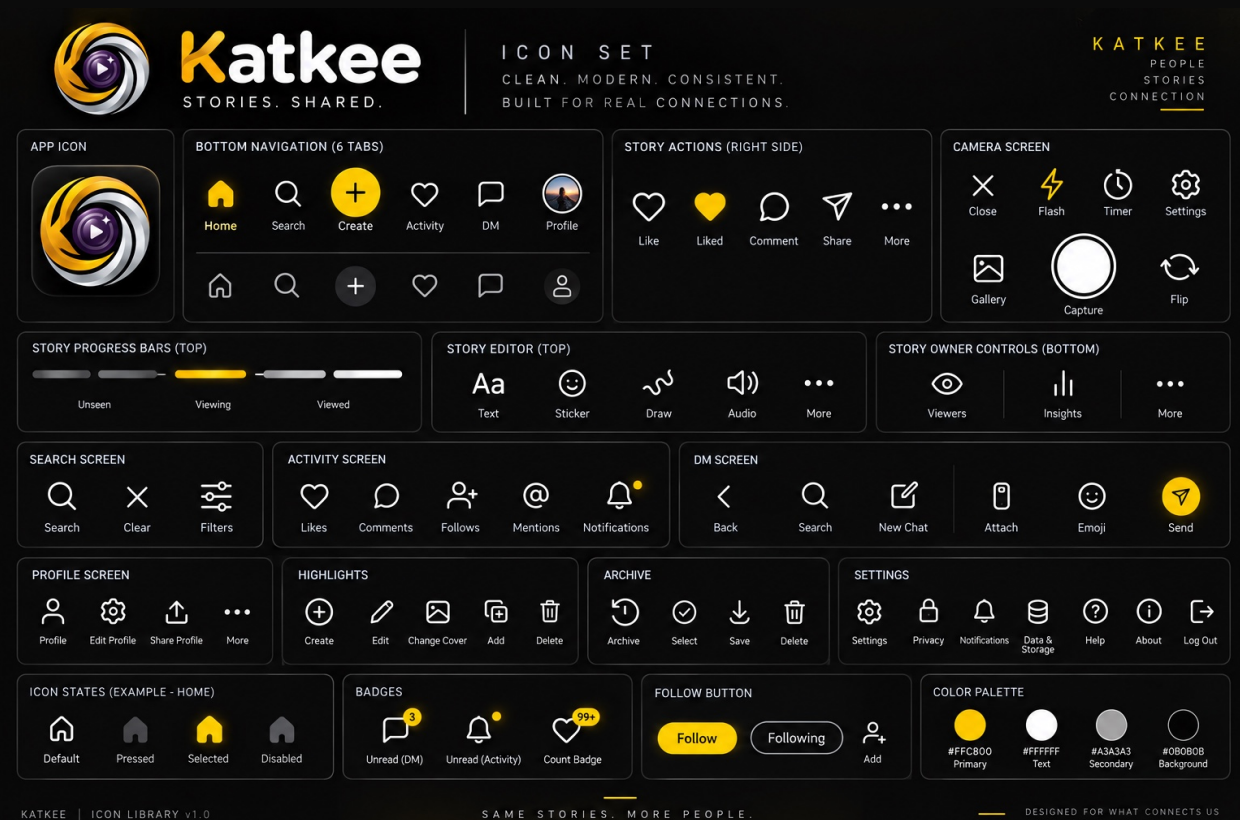
Approved icon system locked

This edition preserves the previously approved Katkee icon library exactly as the visual source of truth. The icon family is not to be redesigned, substituted with another pack, or simplified by Codex.

All product behavior, Story editor tools, Profile/Bio, Highlights, Archive, discovery, DMs, Admin/Super Admin, Sponsored Stories, backend, performance, safety, analytics and QA requirements remain part of the full master specification.

September 2026

1. APPROVED KATKEE ICON LIBRARY - SOURCE OF TRUTH



LOCKED RULE

Use the exact icon design direction shown above for Katkee. Do not invent a replacement icon family. Recreate these as production-ready vector assets while preserving their recognizable geometry, proportions, stroke character, selected states and Katkee color behavior.

Area	Approved behavior
Bottom navigation	Home, Search, Create, Activity, DM, Profile. Activity remains the heart icon shown on the approved board.
Selected state	Katkee yellow/gold. Default is white/soft gray. Pressed/disabled states follow the board.
Create	Yellow circular Create control with dark plus; visually strongest nav action.
Story actions	Like/Liked, Comment, Share, More exactly in this family.
Camera	Close, Flash, Timer, Settings, Gallery, Capture, Flip.
Story editor	Text, Sticker, Draw, Audio/original-audio control, More. No music library or music sticker.
Owner controls	Viewers, Insights, More.
Search	Search, Clear; Filters only when a real people-search filter feature is implemented.
Activity	Likes, Comments, Follows, Mentions, Notifications visual family.
DM	Back, Search, New Chat, Attach if functional, Emoji, Send.
Profile	Profile/Edit Profile, Share Profile, More.
Highlights	Create, Edit, Change Cover, Add, Delete.
Archive	Archive, Select, Save where supported, Delete.
Settings	Settings, Privacy, Notifications, Data & Storage, Help, About, Log Out.
Badges	DM numeric unread, Activity unread dot, count badge capped at 99+.
Follow	Yellow Follow primary; outlined Following; Add-person icon where relevant.

2. ICON IMPLEMENTATION RULES FOR CODEX

Codex must treat the approved board as a design contract, not inspiration for another redesign.

- Create a single reusable vector icon component/library. Prefer SVG/vector paths or the framework's vector system.
- Do not use emoji, Unicode glyphs, mixed third-party icon packs or raster screenshots as functional controls.
- Preserve the approved icon identity across iOS and Android while respecting platform rendering and accessibility.
- Keep optical weight consistent at small sizes. Test at actual device scale, not only enlarged design-board scale.
- Use approximately 44pt iOS / 48dp Android touch targets where appropriate even when the visible glyph is smaller.
- Selected/active icons use #FFC800 yellow/gold. Default icons use white/soft gray on #0B0B0B.
- Do not introduce pink/purple Instagram-style gradients. Purple remains primarily part of the Katkee app-logo identity.
- Activity bottom navigation is the approved heart icon. The Activity screen can use event-specific icons internally.
- Profile selected state may use the user's avatar with a Katkee yellow ring if implemented consistently with the approved board.
- Microanimations are restrained: brief scale/opacity/fill transitions; no continuous pulsing or toy-like bounce.
- Haptics are subtle and reserved for meaningful actions such as Like, Follow, capture/record, send, and drag-drop completion.
- All icon-only controls require accessible labels and states.

The remaining pages contain the complete Katkee product and engineering master specification. Where any older icon illustration conflicts with this locked icon page, THIS APPROVED ICON LIBRARY OVERRIDES IT.

4. Icon Specifications for Codex

Icon	Default	Active/selected	Special state
Home	Rounded roof + inset base outline	Gold outline + subtle warm container	None
Search	Circular lens + clean diagonal handle	Gold lens/handle	Typing does not animate icon
Create	Gold circular/squircle button with dark +	Same; pressed scale only	Recording state belongs to camera, not nav
Activity	Rounded bell/radar-inspired glyph	Gold	Unread dot/badge
DM	Rounded chat bubble + short message line	Gold	Numeric unread badge 1-99+
Profile	Head/shoulders or avatar ring	Gold ring/outline	Active Story ring remains separate
Like	Heart outline	Gold filled/stronger	Double-tap burst around touch point
Comment	Rounded bubble	Gold only when contextually selected	Count below/near control
Share	Three-node/share arrow geometry	Gold when sheet open	Sent confirmation in sheet
More	Three evenly spaced dots	Gold only when menu open	No animation
Text	Aa custom glyph	Gold in text mode	Keyboard active
Sticker	Rounded sticker corner peel	Gold in sticker mode	None
Draw	Pen/brush stroke	Gold in draw mode	Selected tool state
Filter	Overlapping lens/circles	Gold in filter mode	Filter thumbnail selected ring
Mention	@ custom glyph	Gold in mention mode	Search state
Archive	Archive tray/box	Gold selected	Private-only context
Insights	Ascending bars/connection chart	Gold selected	Owner-only

5. Product Promise & Navigation

- Core loop: Stories -> People Discovery -> Connections -> Memories.
- Bottom nav exactly Home · Search · + · Activity · DM · Profile.
- No Reels, Discover, permanent post grid, music, livestreaming, public Story views or circular Highlights.
- Primary intentions should take 0-2 actions where reasonably possible.

6. Home - Full-Screen Story Feed

- One creator per viewport with grouped active Stories.
- Tap right/left within creator; swipe up/down creators; double tap ensures like; hold pauses.
- Top segmented progress; bottom-left identity/Follow/caption; right rail Like/Comment/Share/More.
- Continuous vertical swipe; restore creator/Story/playback context after overlays/screens.
- Poster first; preload next Story + next creator first Story only.

7. Home - Gesture Arbitration

- Single tap waits to distinguish double tap.
- Vertical threshold cancels tap.
- Hold pauses progress/video.
- Rapid taps serialized.
- Buffering keeps poster/last frame.
- First-use hints disappear after success.

8. Home - Comments / Share / More

- Comments bottom sheet pauses Story; real pagination/composer/delete own/report.
- Share bottom sheet: send to user, copy eligible link, native share; privacy enforced.
- More others: Not Interested, Mute, Report, Block.
- More own: Add to Highlight, Insights, Save where supported, Story settings, Delete.

9. Discovery & Ranking

- Eligibility before ranking: privacy/expiration/block/moderation.
- Separate Story Quality and Viewer-Creator Affinity.
- Strong signals: sequence continuation, profile visit, follow, meaningful reply, share, repeat visit.
- Follower count little/no quality weight; new creators get controlled exploration.
- Confidence smoothing + anti-bot/dedup.

10. Search

- People-only search; autofocus + keyboard.
- Suggested for you before typing.
- Search username/display name/interests; inline Follow.
- Debounce, cancellation, pagination, privacy/block enforcement.
- No Explore/trending/Reels grid.

11. Create Camera

- One tap + -> camera; no chooser.
- Tap photo; hold video; release stop; slide up/down while recording for zoom.
- Pinch zoom; double tap flip; tap focus.
- Gallery, flash/timer, front/rear, mic, permissions, zoom indicator.
- Camera preview target <=500ms; p95 <=900ms.

12. Story Editor - Architecture

- Full-screen media; X/back; Text, Sticker, Draw, More; original-audio mute for video.
- No music.
- Normalized overlay coordinates and explicit zIndex.
- GPU/native transforms during gestures; gesture coordinator prevents conflicts.
- Share Story primary action.

13. Editor - Text

- Styles: Clean, Bold, Classic, Modern, Typewriter, Outline; optional original Soft/Highlight.
- Color, alignment, background/no-background, live size, multiline auto-fit.
- Drag, pinch, rotate, double-tap re-edit, delete target, guides/snap.
- Preserve transform on re-edit; published render matches preview.

14. Editor - Caption

- Separate from canvas text.
- Multiline + counter + server-enforced max.
- @ triggers real mention suggestions; structured mention IDs/ranges.
- Draft preserves exact text/line breaks.
- Home 1-2 lines + more; safe rendering.

15. Editor - Mentions

- Sticker/object and caption/text mentions where supported.
- Real debounced people search.
- Immutable target user ID authoritative.
- Deep-link to Profile if access remains allowed.
- Respect blocks/privacy/deletion/permissions/rate limits; notify safely.

16. Editor - Stickers / Emoji / Date / Time / Location

- Bottom sheet; canvas remains visible.
- Emoji, Mention, Location, Date, Time, original Katkee stickers.
- Common move/resize/rotate/delete model.
- Location permission only when chosen; never silently publish precise location.
- No music sticker.

17. Editor - Filters

- Original, Warm, Cool, Bright, Cinema, Mono, Vintage, Soft, Vivid, Night, Fade.
- Non-destructive live preview; optional intensity.
- Filter mode owns horizontal navigation; no overlay conflict.
- Store ID/version/intensity; deterministic export; preserve draft on failure.

18. Editor - Draw

- Pen, Marker, Highlighter, Eraser; size/color/Undo/Redo/Done.
- Draw mode owns gestures; overlay transforms disabled temporarily.
- Smooth normalized strokes; predictable eraser; final output matches preview.

19. Editor - Layers / Drafts / Publish

- Explicit zIndex; new overlay above existing; optional Bring Forward/Send Backward.
- Draft survives exit/crash/network/upload failure.
- Exit with edits -> Keep Editing / Discard.
- Publish: Preparing -> Uploading -> Processing -> Published/Failed.
- Background upload; local draft retained until confirmed; retry preserves edits.

20. Activity

- Today/Earlier; avatar/event/time/optional Story thumbnail.
- Batch repetitive likes; keep important comments/mentions/requests individual.
- Unread indicator + 99+ badge.
- Deep links; not DMS/analytics.

21. DM

- Inbox search/New Message/rows/unread.
- Chat with Story refs, optimistic send, delivery/read/retry, pagination.
- Realtime transport; DB authoritative.
- Server membership auth prevents IDOR.
- No DM content for ad targeting.

22. Profile

- Identity + active Story ring + Follow/Message + Highlights.
- No recent Story/post/Reels grid.
- Display name, username, bio, optional 2-3 tags; follower counts secondary.
- Highlights exactly 3 rectangular/portrait cards per row.

23. Edit Profile / Bio

- Avatar choose/capture/crop/preview/upload; old avatar remains until commit.
- Display name validation.
- Username uniqueness server-side + debounced availability + reserved/profanity rules; immutable user ID preserves links.
- Bio multiline + counter + server max + line breaks + safe rendering.
- Optional limited interests; Save/Discard/error states; propagate changes to all cached surfaces.

24. Highlights

- Named permanent ordered albums; no circles.
- Create: Select Memories -> Customize -> Create.
- Multi-select, proposed/changeable cover, name, drag reorder.
- Direct add from own Story; Edit -> Add Memories.
- Tap opens full-screen viewer; removing item does not delete Archive source.

25. Archive

- Private owner-only memory library grouped by date.
- Tap view; long-press multi-select; Add to Highlight/Save/Delete.
- 24h active expiration separate from Archive/Highlight retention.

26. Story Views & Insights

- Owner-only viewer count/list; never public.
- Views=qualified plays; Viewers=unique eligible accounts.
- Impression vs qualified/duration/complete/replay separate.
- Insights: completion, interactions, Profile Visits, follows, discovery source, repeat visits.
- Prioritize connection metrics.

27. Authentication / Social Graph

- Real signup/login/logout/session/reset/delete.
- Follow/unfollow, private requests, block/mute.
- Secure sessions; server-side privacy/authorization; signed media where needed.

28. Settings / Notifications

- Account, Privacy, Notifications, Data & Storage, Language, Help, About, Logout, Delete Account.
- Notifications prioritize replies/follows/mentions/comments/requests; batch noise; no guilt/fake urgency.

29. Admin / Super Admin

- Separate protected web console; shared backend.
- USER/ADMIN/SUPER_ADMIN RBAC; MODERATOR-ready.
- Super Admin manages Admins/permissions.
- Admin acts only within granted permissions.
- MFA-ready privileged sessions/re-auth/rate limits; no client-only admin auth.

30. Moderation / Appeals / Audit

- Queue by type/status/reason/priority.
- Review content, account, report context, history, notes.
- Keep/Remove/Restrict/Suspend by permission.
- Removal immediate from public surfaces; evidence retained by policy.
- Immutable audit; concurrent review protection; appeals; no unrestricted DM browsing.

31. Sponsored Stories

- Full-screen clearly labeled Sponsored units in Home; no Ads tab.
- Brand + media + CTA + Hide/Report/Why.
- Organic ranking separate from ad delivery.
- Configurable caps/minimum organic gap; no consecutive aggressive ads.
- Approved active eligible creatives only; failure skips to organic.

32. Ads Campaign Backend

- V1 Admin-managed flow: Details -> Creative -> Audience -> Budget/Schedule -> Preview -> Review.
- Statuses DRAFT/PENDING_REVIEW/APPROVED/ACTIVE/PAUSED/REJECTED/COMPLETED.
- Only broad non-sensitive targeting; no DM content/sensitive traits.
- Track requested/rendered/impression/qualified view/complete/click/hide/report.

33. Core Data Domains

Domain	Entities
Identity	users, profiles, sessions, device_tokens
Social	follows, follow_requests, blocks, mutes
Stories	media, stories, views, likes, comments, comment_likes
Memories	highlights, highlight_items
Messaging	conversations, members, messages, receipts
Activity	notifications
Safety	reports, moderation_cases/actions, appeals
Admin	roles, permissions, audit_logs
Discovery	events, creator_affinity, ranking_features
Ads	advertisers, campaigns, creatives, targeting, delivery/events, reviews

34. Media Pipeline

```
client -> secure upload -> object storage -> validation -> processing -> poster/thumb/video variants ->
CDN/authorized delivery
```

- Validate MIME/size/duration/dimensions/format.
- Photos: orientation/crop/filter/overlays/compression/thumb.
- Videos: orientation/filter/overlays/original audio/transcode/poster/variants.
- No large media binaries in normal DB rows.
- Server created_at/expires_at; 24h active Story expiration; Archive/Highlight retention separate.

35. Performance Release Budgets

Metric	Gate
Cold start usable	<=800ms target; p95 <=1.5s
Warm start	<=300ms; p95 <=600ms
Home cached poster	<=300ms; p95 <=700ms
Video moving frame	<=500ms; p95 <=1s
Story/creator response	<=100ms; p95 <=200ms
Camera preview	<=500ms; p95 <=900ms
Direct manipulation	60 FPS min; p95 60Hz frame <20ms; jank <5%
Publish healthy	<=3s target; p95 <=6s; non-blocking
Read API	p50 <=150ms; p95 300-500ms
Write API	p95 <=700ms
Search	p95 <=500ms after debounce
DM local / ack	<=50ms local / <=300ms target

36. Product Health Targets

Area	Metric	Target
Reliability	Crash-free sessions	>=99.8%
Retention	D1	Aim >=35%; strong >=45%
Retention	D7	Aim >=20%; strong >=25%
Retention	D30	Aim >=10-15%; strong >=18%
Discovery	New eligible users see >=5 viable creators	>=90% once supply permits
Discovery	Qualified recommendation views	Track toward >=60%
Discovery	Profile visit after discovery	Track toward >=8-12%
Discovery	Follow after profile visit	Track toward >=15-25%
Safety	High-priority review	<1h when staffed
Safety	Normal review	<24h
Safety	Block/report action success	>=99.9%
Ads	Post-ad organic continuation	No material degradation vs control

37. Analytics Events

- creator_impression, story_impression, qualified_view, watch_duration, story_complete, story_replay
- story_next/previous, creator_sequence_started/continued/completed, creator_swipe_next/previous, quick_creator_skip
- story_like/unlike, comment_open/created/deleted, story_share, profile_visit, follow_after_story, repeat_creator_visit
- not_interested, mute, block, report, story_reply_sent, conversation_started
- highlight_created/item_added/reordered, ad_requested/rendered/impression/qualified_view/complete/click/hide/report

Version event schemas, dedupe/idempotency, batch asynchronously and never block primary interactions.

38. Safety / Security / Accessibility

- Protect against IDOR, privilege escalation, bots, fake accounts/follows/views, mass comments/DMs, impersonation, malicious links, report abuse.
- Safety eligibility before recommendation; removed content ineligible immediately.
- Screen-reader labels, large targets, focus order, contrast, reduced motion, dynamic text where practical.
- No color-only meaning; drag/reorder accessible alternatives.
- Rate limits, transactions and idempotency for sensitive operations.

39. Loading / Empty / Offline / Error

Surface	Required states
Home	loading poster, caught up, network retry without context loss
Search	suggested/no results/loading/error
Camera	permission denied/unavailable hardware
Editor	tool modes/draft restore/export error
Publish	preparing/uploading/processing/waiting/failed/retry
Activity	empty/loading/error
DM	empty/offline/send failed/retry
Profile	no Story/no Highlights/error
Admin	empty/permission denied/conflict/error
Ads	no eligible/failed creative -> organic fallback

40. Codex Implementation Order

Phase	Deliverable
1	Repo audit + baseline
2	Premium icon assets/design tokens/nav
3	Auth/Profile/Edit Bio/social/privacy
4	Camera/gallery
5	Full editor: text/caption/mentions/stickers/draw/filters/layers/drafts
6	Story lifecycle/viewer
7	Home gestures/interactions/context
8	Discovery/events/exploration
9	Search/Activity
10	DM/Story replies
11	Archive/Highlights/reorder
12	Viewer lists/Insights
13	Admin/moderation/audit/appeals
14	Ads backend/Admin tools
15	Sponsored Story insertion/caps/hide/report
16	Security/accessibility/performance
17	Full real-device regression/release

41. Screen-by-Screen Acceptance Inventory

- Auth: Welcome/Signup/Login/Reset/Session recovery.
- Home: Story viewer/hints/Comments/Share/More/Report/Block/Mute/Not Interested/caught-up/error.
- Search: landing/Suggested/results/no-results/follow states.
- Create: Camera/Gallery/permissions/record/zoom.
- Editor: idle/Text/Caption/Mentions/Stickers/Emoji/Location/Date/Time/Filters/Draw/Layers/Discard/Settings/Publish/failure/draft restore.
- Activity: Today/Earlier/batched event/follow request.
- DM: Inbox/Search/New Message/Chat/Story reference/send failure/More.
- Profile: Own/Other/Edit Profile/Avatar crop/Username validation/Bio/Interests/Followers/Following/More.
- Story owner: Viewer list/Insights.
- Highlights: grid/select/customize/cover/view/edit/add/reorder.
- Archive: grouped library/multi-select/view.
- Settings: Account/Privacy/Notifications/Data/Language/Help/About/Logout/Delete.
- Admin: Login/Dashboard/Reports/Review/Users/History/Appeals/Admins/Permissions/Audit.
- Ads: Advertisers/Campaigns/Create/Creative/Audience/Budget/Preview/Review/Analytics.

42. Final Paste-Ready Codex Directive

KATKEE PREMIUM 10/10 DIRECTIVE

Audit the existing repository first. Preserve working behavior and data.
Use migrations and implement this PDF additively.

PREMIUM ICON SYSTEM:

Build a Katkee-original vector icon family with rounded geometry, consistent stroke weight, deliberate negative space and dark-mode optical tuning.
Default = soft white/gray. Selected = Katkee gold/yellow with restrained warm state container. Violet is an interaction/brand accent, not the default color for every control. Create is the strongest nav control.
Do not use emoji, Unicode symbols, random icon packs or raster screenshots as production controls.

NAV = Home | Search | + | Activity | DM | Profile

HOME:

Full-screen Stories. Tap left/right same creator. Swipe up/down creator.
Double tap ensures like. Hold pauses. Preserve return context.

EDITOR:

Text + Caption + Mentions + Stickers/Emoji/Location/Date/Time + Filters + Draw + Layers + original audio + durable Drafts + reliable Publish.
NO MUSIC.

PROFILE:

Active Story via avatar ring only. Edit avatar/display name/username/bio/tags.
Identity + Follow/Message + 3-column rectangular Highlights. No post grid.

DISCOVERY:

Optimize for whether viewer wants to see the PERSON again. New creators get controlled exploration. Safety eligibility before ranking.

ADMIN:

Separate protected web console. USER/ADMIN/SUPER_ADMIN server RBAC.
Moderation + appeals + immutable audit. No unrestricted DM browsing.

ADS:

Clearly labeled Sponsored Stories in Home only. Separate organic ranking from ad delivery. Approval, caps, organic gaps, hide/report/why, safe fallback.

DONE =

functional + persistent + secure + recoverable + instrumented + accessible + performance-verified + real-device tested.

After each phase: BUILD -> TEST -> REGRESSION -> FIX -> CONTINUE.

43. Final 10/10 Release Gate

Gate	Must be true
Visual	Premium icon family consistent everywhere; no basic/mixed glyphs.
UX	Each tab has one obvious job; common intent <=2 actions where practical.
Speed	Home/camera/editor budgets measured.
Creation	All editor tools + drafts reliable.
Discovery	Emerging creators can earn distribution.
Connection	Story -> Profile -> Follow/Reply -> DM loop works.
Memories	Archive/Highlights lifecycle + ordering persist.
Safety	Block/report/moderation/RBAC/audit/appeals tested.
Ads	Clear, capped, safe, non-blocking.
Security	IDOR/privilege/privacy tests pass.
Accessibility	Labels/contrast/targets/reduced motion/reorder alternatives.
Trust	No fake engagement/social proof.
Regression	Existing Katkee flows still pass.

END - KATKEE PREMIUM ICONS + FULL MASTER SPEC

44. Performance Architecture - Speed Is a Product Feature

Instant first frame. Everything else just in time. Never make the user wait for work that can happen asynchronously.

- Katkee must feel immediate even when the network is imperfect. The critical path is Home first frame, creator swipe, camera preview and Story editor interaction.
- Do not download the whole feed. Fetch a small ranked batch, show a poster immediately, prioritize current media, then preload only the most likely next media.
- Do not initialize Archive, Insights, Admin, full DM history, ad-management code or other non-critical features before Home becomes usable.
- Measure p50, p95 and p99 in production by platform, app version, device class and network class. Averages are not sufficient.

45. Home Feed - Just-in-Time Loading

- On app open: restore lightweight session/context, render Home shell, show cached/current poster, then begin current video playback.
- Fetch only a bounded first batch of creator/story metadata. Paginate before the user reaches the end.
- Priority 1: current Story media. Priority 2: next Story from the same creator. Priority 3: first Story of the next creator. Priority 4: useful previous Story for back navigation.
- Cancel or deprioritize media requests when the viewer swipes past content.
- Keep a bounded memory cache for immediate neighbors and a disk cache for reusable posters/thumbnails/recent media.
- Never preload dozens of full videos. Adapt preload depth to memory pressure, network quality and device capability.
- Returning from Comments/Profile/Share/DM should restore creator ID, Story index and playback position without rebuilding the feed.

Preferred startup path: Open Katkee -> cached poster -> current video -> next Story -> next creator first Story.

46. Video Delivery - Never Stream Raw Originals

Upload -> object storage -> background media processing -> optimized variants + poster -> CDN/edge delivery

- Do not serve the user's original uploaded video as the normal Home playback asset.
- Generate appropriate playback variants such as 360p, 540p, 720p and 1080p when source quality supports them.
- Generate poster frame and thumbnail during processing.
- Select an initial rendition based on device/network conditions; prioritize quick playback start over unnecessarily high initial resolution.
- Use streaming/range-friendly delivery and a player that can recover from transient network changes.
- Keep audio/video sync and original recorded audio intact.
- Version processed media so cache invalidation is deterministic.

47. CDN, Caching & Media URLs

- Story media, avatars, thumbnails and poster frames should be delivered through CDN/edge infrastructure rather than the application API server streaming bytes.
- The API returns metadata and authorized media references; media delivery is a separate optimized path.
- Cache immutable/versioned media aggressively. Use versioned keys or URLs for changed avatars/media derivatives.
- Cache avatars, poster frames, thumbnails, profile basics and recently viewed media with bounded policies.
- Private media must still honor authorization; use signed/authorized delivery where required.
- Do not let stale profile caches survive indefinitely after username/avatar/bio changes.
- Track CDN cache hit rate and origin egress so performance and cost can be tuned together.

48. Story Upload - Fast Perceived Publishing

- Share Story should return control quickly. Do not block the user on full server processing.
- Prepare/compress intelligently on-device without causing long main-thread stalls.
- Upload directly to secure object storage where architecture permits, then process asynchronously.
- Keep the local draft until backend publication is confirmed.
- States: Preparing -> Uploading -> Processing -> Published; Offline -> Waiting for connection; Failure -> Retry without losing edits.
- Resume interrupted uploads where practical. Use idempotency so retries cannot create accidental duplicate Stories.
- Multiple uploads should use a subtle queue instead of multiple blocking modals.
- Others must not see a Story until the backend marks it publishable.

49. Analytics & Social Actions - Never Block Interaction

- Likes, follows, read states and safe reversible actions should use optimistic UI where appropriate, followed by server reconciliation.
- Story views, watch duration, sequence signals, recommendation events and ad events are queued/batched asynchronously.
- Do not perform: user watches -> analytics request -> wait -> continue.
- Preferred path: user watches -> UI/playback continues immediately -> analytics queue flushes independently.
- Persist critical pending analytics enough to avoid obvious loss, but never at the cost of janking playback.
- Deduplicate event retries and use stable event IDs/idempotency where needed.

50. Recommendation Serving - Keep Ranking Off the Swipe Path

- Do not compute a large expensive recommendation universe from scratch every time the user swipes to the next creator.
- Maintain candidate pools and precomputed/cacheable creator/story features where reasonable.
- Perform eligibility and safety checks before final ranking.
- Return small ranked batches with a continuation cursor and refresh before the local queue runs dry.
- Keep organic ranking separate from Sponsored Story insertion.
- Invalidate or filter deleted, expired, blocked, muted or moderation-removed content immediately.
- Recommendation logging and model-feature updates must not delay Story transitions.

51. Story Editor - 60 FPS Interaction Architecture

- Move/scale/rotate Text, Mentions, Stickers and other overlays using native/GPU/compositor transforms during active gestures.
- Do not rerender the entire editor component tree for every pointer movement.
- Commit normalized x/y/scale/rotation/zIndex values when a gesture ends or at a controlled cadence.
- Drawing should coalesce/simplify pointer samples without visibly degrading the stroke.
- Filter previews should use optimized GPU/native paths where possible and must not block overlay interaction.
- Move media export/compositing off the UI thread.
- Profile editor frame time and jank on representative mid/lower Android hardware, not only flagship devices.

52. Database & API Performance

- Eliminate N+1 patterns in Home, Search, Activity, DM, Profile and Insights.
- Home response should include the compact creator/story metadata needed to render the viewport without a chain of dependent requests.
- Use pagination/cursors for Stories, comments, search, notifications, messages, viewers and admin queues.
- Index high-frequency filters/sorts such as story creator + expiration/state, follow edges, blocks, conversation membership, message timestamp, notification recipient/read state, highlight ordering and moderation status.
- Keep counts/materialized summaries where justified instead of repeatedly scanning large event tables.
- Use connection pooling and bounded query timeouts. Investigate slow-query logs.
- API payloads should be compact; avoid sending editor-only/admin-only fields to Home.
- Compress textual API responses where appropriate.

53. Startup & Feature Loading

- Keep the startup bundle/path small. Load only what is required to authenticate/restore session and render the initial consumer surface.
- Lazy-load heavy editor, Archive, Insights, Admin and ad-management modules when they are actually opened.
- Defer non-essential SDK initialization until after first usable UI where safe.
- Avoid synchronous startup migrations, large JSON parsing, expensive image decoding or recommendation computation on the main thread.
- Warm-start should reuse valid local state and avoid unnecessary refetch/reconstruction.
- Track startup spans so regressions can be attributed to specific initialization stages.

54. Memory, Battery & Network Discipline

- Bound player instances and decoded video buffers. Release media resources for distant Stories.
- Pause/stop playback when Katkee backgrounds or another full-screen media surface takes ownership.
- Respect OS memory warnings and reduce preload depth on constrained devices.
- Do not poll aggressively for DMS/Activity when realtime/push mechanisms are available.
- Batch background work and analytics to reduce radio/battery churn.
- Adapt video quality/preload on constrained networks rather than repeatedly stalling at high quality.
- Prevent runaway disk cache growth with size/age limits and eviction.

55. Performance Release Budgets

Interaction / metric	Target	Release concern
Cold start -> first usable UI	<=800ms target; p95 <=1.5s	Investigate/block if routinely >2.5s
Warm start	<=300ms; p95 <=600ms	Regression if materially worse
Home cached poster	<=300ms; p95 <=700ms	Critical
Video first moving frame	<=500ms; p95 <=1s	Critical; >2s is severe
Tap -> next Story visual	<=100ms; p95 <=200ms	Critical
Swipe -> next creator visual	<=100ms; p95 <=200ms	Critical
Camera usable preview	<=500ms; p95 <=900ms	Critical
Photo capture feedback	<=100ms; p95 <=200ms	Critical
Editor direct manipulation	60 FPS minimum	p95 60Hz frame <20ms; jank <5%
Share -> control returned	<=300ms; p95 <=600ms	Must remain non-blocking
Healthy Story publication	<=3s target; p95 <=6s	App remains usable
Normal read API	p50 <=150ms; p95 300-500ms	p99 <=1s target
Write API	p95 <=700ms	Optimistic UI where safe
Search API	p95 <=500ms after debounce	Cancel stale responses
DM local bubble	<=50ms	Immediate perceived send
DM server acknowledgement	<=300ms target; p95 <=600ms	Retry if failed

56. Performance Telemetry Dashboard

- Track startup cold/warm, Home poster time, first moving frame, rebuffer rate, Story transition latency, creator swipe latency, camera preview time, editor frame time/jank, upload duration/failure/retry, API p50/p95/p99 and DM acknowledgement.
- Segment by iOS/Android, OS version, app version, device performance tier, network type/quality and geography/region where privacy policy permits.
- Track media rendition selected, CDN cache hit/miss, bytes delivered per Story, abandoned-before-play and rebuffer duration.
- Create regression alerts for meaningful p95/p99 degradation after releases.
- Performance dashboards should distinguish backend latency, media/CDN latency and client rendering latency.

57. Low-End Device & Constrained-Network Test Matrix

Scenario	Must verify
Representative iPhone	Cold/warm launch, Home, camera, editor, upload, DM
Mid-range Android	All critical paths + sustained Story swiping
Lower-end supported Android	Memory pressure, decode, editor gestures, camera preview, upload
Good Wi-Fi	Baseline
Good mobile data	Normal mobile playback/upload
Constrained mobile	Adaptive quality, poster-first, reduced preload, retry
High latency / packet loss	No frozen UI; graceful buffering and retries
Offline -> reconnect	Draft/upload recovery; Home error state preserves context
Warm cache / cold cache	Both measured separately
Long session	No memory growth/player leaks/cache runaway

58. Codex Performance Implementation Checklist

- Profile the current app before changing architecture; record baseline numbers.
- Instrument the critical spans first so improvements can be measured.
- Implement poster-first Home rendering and bounded preloading.
- Implement CDN-first processed media variants; stop serving raw originals for normal playback.
- Add bounded memory/disk caches and deterministic invalidation.
- Move analytics/event delivery off interaction-critical paths.
- Optimize recommendation batch serving and prefetch.
- Audit DB indexes and N+1 queries using real query plans/slow logs.
- Profile camera open and remove nonessential startup work.
- Profile Story editor frame time; move transforms/compositing off broad UI rerenders.
- Implement resumable/retry-safe Story uploads and idempotent publish.
- Lazy-load noncritical features/modules.
- Add p50/p95/p99 production dashboards and release regression alerts.
- Run the full device/network matrix before release.

59. Performance Acceptance Gate for Katkee

The four hardest release gates are: Story first-frame time, creator-swipe smoothness, camera preview time, and editor frame time.

- Do not approve a release because the app feels fast on a flagship developer phone.
- Do not approve if Home routinely shows blank media while waiting for video.
- Do not approve if swiping creators causes network-dependent UI stalls.
- Do not approve if editor drag/pinch/rotate visibly janks on supported mid-range hardware.
- Do not approve if Share Story traps the user behind upload/processing.
- Do not approve if analytics, ad tracking or recommendation logging blocks playback.
- Do not approve if memory usage grows continuously during a long Story session.
- Performance work must preserve privacy, moderation, authorization and product behavior.

ENGINEERING PRINCIPLE: Instant first frame. Everything else just in time.

60. Authentication Upgrade - Google + Phone Login

Katkee must support fast, secure account entry through Google and mobile phone OTP, while preserving the existing account/session model and preventing duplicate identities.

- Keep any existing supported login method unless explicitly deprecated through a safe migration.
- Add Continue with Google and Continue with Phone as first-class authentication options.
- Both methods must create or authenticate the same canonical Katkee user/account model. Do not create separate profile systems for each provider.
- Authentication provider identity is linked to an immutable internal Katkee user ID.
- Login should be fast enough that authentication does not become a noticeable startup bottleneck after a valid session already exists.
- Returning users with a valid refresh/session token should not be forced through login again.

61. Welcome / Login Screen UX

Element	Behavior
Katkee branding	Approved app logo/wordmark; dark Katkee visual system.
Continue with Google	Prominent provider button using the official Google sign-in mark according to provider branding requirements.
Continue with Phone	Katkee-styled primary/secondary button that opens phone entry.
Existing email/password	Keep only if already supported/product-approved; do not silently remove working accounts.
Terms / Privacy	Compact links before account creation/sign-in consent where required.
Loading	Button-local progress; do not freeze the whole screen.
Error	Human-readable inline message + retry; preserve user input.

- Do not add unnecessary onboarding screens before authentication.
- Do not ask for profile details that can be completed after successful authentication.
- Never fake a successful provider login in development builds that claim production readiness.

62. Google Sign-In Flow

Tap Continue with Google -> provider account selection/authorization -> verified provider result -> Katkee backend verification -> account link/create -> session -> onboarding or Home.

- Use the platform/provider-supported OAuth/OpenID Connect flow and official SDK/mechanism appropriate to the Katkee stack.
- Never trust only a client-provided Google user ID/email. The backend must verify the provider token/assertion and its intended audience/client.
- Use provider subject/identity ID as the stable external identity; email is profile/contact data and can change.
- Existing linked Google user -> sign in to the same Katkee account.
- New Google user -> create canonical Katkee account, then complete only missing required onboarding fields such as unique username.
- If Google returns a verified email and Katkee already has an account with that email, do not automatically merge identities unless the merge/link policy proves ownership safely.
- Handle user cancellation, provider failure, revoked authorization and network failure without corrupting session state.
- Store only tokens/secrets that are actually required, using secure platform/server storage. Never log raw provider tokens.

63. Phone Login - Number Entry

- User chooses country/region code and enters mobile number.
- Normalize to a canonical international format such as E.164 before backend use.
- Validate basic format client-side for UX, but server remains authoritative.
- Show the normalized/masked destination before sending OTP.
- Send Code button enters a sending state and cannot be spam-tapped.
- Do not reveal whether a phone number belongs to a particular person/account through overly specific public error messages.
- Respect supported-country/product policy rather than assuming every numbering plan/SMS route is available.
- Phone number is private account data and must not be publicly exposed on Profile unless a separate explicit product feature exists.

64. Phone OTP Verification

Phone number -> request OTP -> enter code -> server/provider verifies -> canonical Katkee account -> secure session.

- Use a reputable OTP/SMS authentication provider or the existing backend auth provider's phone authentication capability.
- OTP codes must be short-lived and single-use.
- Verification must happen server/provider-side; never accept a client-only 'verified' flag.
- Support automatic code fill where the platform securely provides it, but always allow manual entry.
- Use a clear 6-digit-style code UI only if that matches the configured provider; do not hard-code a code length that the provider does not guarantee.
- Show Resend Code only after a cooldown. Display a countdown without trapping the user.
- Allow Edit Number to return without losing the auth flow.
- Expired/wrong code -> clear inline error; do not wipe the phone number.
- Successful verification invalidates the challenge so replay cannot create another session.

65. OTP Abuse, Fraud & Cost Protection

- Rate-limit OTP requests by phone number, account/session/device signals and network/IP risk signals where appropriate.
- Apply resend cooldowns and rolling request limits.
- Rate-limit verification attempts; expire or invalidate a challenge after excessive failures.
- Add bot/abuse protection or step-up challenge when risk is high rather than challenging every normal user unnecessarily.
- Monitor SMS pumping/fraud patterns, unusual country/operator spikes and repeated requests that never verify.
- Set provider spend alerts/limits where available so OTP abuse cannot create uncontrolled SMS cost.
- Do not expose exact anti-abuse thresholds in the client.
- Log security events without logging the OTP code itself.

66. Account Linking & Duplicate Prevention

- One Katkee user can have multiple authentication identities linked to the same internal user ID when product policy allows it.
- Recommended identity model: internal user_id + provider type + provider subject/phone identity + timestamps + verified state.
- Never create a second Katkee profile merely because an existing user later adds Google or Phone.
- Linking a new provider requires proof of the currently authenticated account plus successful verification of the new provider.
- Before linking, check whether the new provider identity is already linked to another Katkee account. If so, stop and use a protected recovery/merge path rather than silently moving it.
- Unlinking must never leave an account with no usable sign-in/recovery method unless deletion is the intended action.
- Username, followers, DMs, Stories, Highlights and moderation history remain attached to the internal Katkee user ID, not to Google email or phone number.

67. New-User Onboarding After Google / Phone

- After first successful authentication, ask only for missing essentials.
- Suggested sequence: unique username -> display name/avatar optional or prefilled where permitted -> a few non-sensitive interests -> optional suggested follows -> Home.
- Google profile name/avatar may be offered as a starting value where provider permissions and product policy allow, but the user can edit them.
- Phone login must not force contact syncing.
- Do not require the user to follow people before seeing Home; Katkee discovery must solve the empty-network problem.
- Keep onboarding resumable so an app interruption does not create an unusable half-account.

68. Session, Logout & Reauthentication

- After successful Google/Phone authentication, issue/use the same Katkee session model as other login methods.
- Store client session credentials only in secure platform storage.
- Refresh sessions securely; revoke on logout/account security actions as supported.
- Logout clears local sensitive session state while preserving safe public caches only where appropriate.
- Critical account actions such as changing linked phone/provider, deleting account or privileged Admin actions may require recent reauthentication.
- Account deletion must clean/revoke authentication identities according to backend/provider requirements and Katkee retention policy.
- Blocked/suspended/disabled account state is checked server-side after authentication; valid provider credentials do not bypass Katkee enforcement.

69. Authentication Data Model

```
users
id (immutable Katkee user ID)
account_state
created_at / updated_at

auth_identities
id
user_id -> users.id
provider_type: GOOGLE | PHONE | EMAIL? ...
provider_subject / canonical_phone_identity
verified_at
created_at / updated_at
last_used_at
UNIQUE(provider_type, provider_subject_or_phone)

profiles
user_id -> users.id
username
display_name
bio
avatar_media_id
...
```

- Adapt names to the existing Katkee schema instead of creating parallel tables if equivalent auth identity structures already exist.
- Use database uniqueness constraints in addition to application checks.
- Do not store OTP plaintext in Katkee tables if the auth provider owns OTP verification.
- Phone/email fields containing personal data should have appropriate access controls and logging discipline.

70. Authentication API / Backend Contract

Capability	Backend responsibility
Google exchange/verification	Verify provider assertion/token, resolve identity, create/link user safely, issue Katkee session.
Request phone OTP	Normalize/validate, abuse checks, request provider challenge, return opaque challenge state.
Verify phone OTP	Validate challenge/code with provider, resolve/create user, issue session.
Link Google	Require authenticated/recent session + verified Google identity; prevent cross-account collision.
Link phone	Require authenticated/recent session + successful OTP; prevent collision.
Unlink provider	Authorization + recent auth; ensure another usable login remains.
Session refresh	Rotate/refresh according to backend auth design.
Logout/revoke	Invalidate session/refresh material as supported.
Account status	Enforce ACTIVE/RESTRICTED/SUSPENDED/DISABLED rules server-side.

71. Authentication Performance Targets

Metric	Target / expectation
Valid existing session -> Home	Must not wait for interactive Google/Phone flow.
Login screen usable	Fast local render; provider SDK initialization should not block unrelated UI.
Google handoff return -> Katkee processing	Aim for <=1s backend/app processing excluding provider/user interaction/network variance.
OTP request API	Aim p95 <=1s excluding external SMS delivery time.
OTP verification API	Aim p95 <=1s excluding provider/network variance.
Post-auth onboarding transition	Immediate UI transition after server-confirmed session.
Auth error recovery	No app restart required.

External provider and SMS delivery times are not fully controlled by Katkee; measure Katkee-owned processing separately from provider latency.

72. Authentication Analytics & Observability

- Track `auth_screen_view`, `google_login_started/succeeded/cancelled/failed`, `phone_login_started`, `otp_requested`, `otp_request_failed`, `otp_verified`, `otp_failed`, `auth_account_created`, `auth_account_linked` and `onboarding_completed`.
- Never include OTP codes, raw OAuth tokens or unnecessary full phone numbers in analytics.
- Track conversion and latency by auth method, platform, app version and failure category.
- Alert on sudden OTP request spikes, provider verification failures, callback/configuration failures and abnormal account-creation duplication.
- Distinguish user cancellation from technical failure so product metrics remain meaningful.

73. Authentication QA & Acceptance

Flow	Must pass
New Google user	Provider -> account -> username/onboarding -> Home.
Returning Google user	Same Katkee account/data restored.
Google cancel/failure	Safe return, no ghost session/account.
New phone user	Number -> OTP -> account -> onboarding -> Home.
Returning phone user	Same account/data restored.
Wrong OTP	Inline error; retry within limits.
Expired OTP	Clear state + controlled resend.
OTP spam attempts	Rate limits/abuse controls activate safely.
Provider linking	Existing user links Google/Phone without duplicate profile.
Collision	Already-linked identity cannot be stolen/moved silently.
Logout/login	Session revoked/cleared; next login restores same account.
Suspended account	Provider auth cannot bypass Katkee suspension.
Offline/provider outage	Recoverable error; no corrupted auth state.
Account deletion	Auth identities/session handling follows deletion policy.
Accessibility	Buttons/fields/code entry/errors labeled and keyboard navigable.

74. Codex Authentication Implementation Directive

ADD GOOGLE + PHONE AUTH WITHOUT BREAKING EXISTING KATKEE AUTH.

1. Audit current auth/session/profile schema and provider configuration first.
2. Reuse the canonical users/profiles/session model.
3. Add Google sign-in through the official supported OAuth/OIDC mechanism.
4. Backend verifies provider identity; never trust client-only IDs/emails.
5. Add phone OTP using the existing/reputable auth provider.
6. Normalize phone numbers; server/provider verifies OTP.
7. Add resend cooldowns, rate limits, anti-bot/fraud controls and spend monitoring.
8. Link auth identities to immutable Katkee user_id.
9. Prevent duplicate profiles and unsafe automatic account merges.
10. Support secure provider linking/unlinking with recent reauthentication.
11. Keep returning valid sessions fast; do not show login unnecessarily.
12. New users complete only missing onboarding essentials, then enter Home.
13. Preserve Stories, follows, DMs, Highlights, reports and all user data across login-method changes because those belong to Katkee user_id.
14. Add auth analytics without OTP/raw token/full-phone leakage.
15. Test new/returning/cancel/error/expired/rate-limit/collision/logout/suspension and account-deletion paths on real iOS and Android.
16. Do not remove existing login methods without an explicit migration plan.

DONE = secure + persistent + duplicate-safe + abuse-resistant + fast + tested.